

# Reviewer Pack — Minimal Grothendieck-Witt Class Rank and Frege Proof Depth C...

Entry 7b09e71afd40 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 16:10:37 UTC

## Minimal Grothendieck-Witt Class Rank and Frege Proof Depth Correlation

**Entry ID:** 7b09e71afd40 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 16:10:37 UTC

### 1. Conjecture

**Field A** (mathematical branch): Algebraic Geometry (Grothendieck-Witt Theory) **Field B** (complexity object): Frege Proof Complexity

#### Statement:

For every CNF  $\varphi$  with  $n$  variables, the minimal Grothendieck-Witt class rank ( $\text{gwc\_rank}(\varphi)$ ) of its clause-indicator polynomial is linearly correlated with its Frege proof depth ( $\text{fp\_depth}(\varphi)$ ), such that  $\text{gwc\_rank}(\varphi) = \Theta(\text{fp\_depth}(\varphi))$ .

#### Rationale (proposer's reasoning):

Grothendieck-Witt theory provides a way to study polynomials over the Boolean ring, and its invariants could potentially capture the complexity of reasoning processes like Frege proofs. If this conjecture holds, it would suggest a new connection between algebraic geometry and proof complexity that has not been explored before.

**Taxonomy category:** AlgebraicGeometry (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 1d7b1bf9ae51c98e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

For a CNF  $\varphi$  with  $n$  variables, we consider the conjecture supported if Pearson's correlation coefficient between  $\text{gwc\_rank}(\varphi)$  and  $\text{fp\_depth}(\varphi)$  is  $\geq 0.8$  for at least 75% of seeds, and we consider it falsified if any seed produces a correlation coefficient  $< 0.5$  or any seed yields a  $\text{gwc\_rank}(\varphi) > 2 * \text{fp\_depth}(\varphi)$ .

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | UNCERTAIN | 1.00       | UNCERTAIN        | HITS             |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - Minimal Grothendieck-Witt class rank AND Frege proof complexity - Clause-indicator polynomial AND Grothendieck-Witt theory AND Frege proof depth - Algebraic geometry AND Frege proof complexity INCLUSIVE gwc\_rank 0 fp\_depth

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def clause_indicator_polynomial(cnf):
        n = len(cnf[0])
        poly = [[0] * (1 << n) for _ in range(len(cnf))]
        for i, clause in enumerate(cnf):
            mask = 0
            for lit in clause:
                if lit > 0:
                    mask |= 1 << (lit - 1)
                else:
                    mask |= 1 << (-lit - 1)
            poly[i][mask] = 1
        return poly

    def grothendieck_witt_class_rank(poly):
        n = len(poly[0])
        rank = 0
        for i in range(1 << n):
            if any(poly[j][i] == 1 for j in range(len(poly))):
                rank += 1
        return rank

    def frege_proof_depth(cnf):
        # Placeholder function; actual implementation depends on the Frege proof system
        return len(cnf) * 2 # Simplified example

    n = random.choice([5, 10, 15, 20, 30, 40])
```

```

cnf = []
for _ in range(n):
    clause = [random.randint(1, n), -random.randint(1, n)]
    cnf.append(clause)

poly = clause_indicator_polynomial(cnf)
gwc_rank = grothendieck_witt_class_rank(poly)
fp_depth = frege_proof_depth(cnf)

return {
    "metric_name": "gwc_rank_vs_fp_depth",
    "metric_value": gwc_rank / fp_depth,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": False if gwc_rank > 2 * fp_depth else True,
    "counterexample": "" if gwc_rank <= 2 * fp_depth else f"gwc_rank({gwc_rank}) > 2 * fp_depth"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"gwc_rank > 2 * fp_depth\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_81f7e84c.py", line 70, in <module>
    trial_result = run_trial(seed)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_81f7e84c.py", line 52, in run_trial
    poly = clause_indicator_polynomial(cnf)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_81f7e84c.py", line 31, in clause_indicator_polynomial
    poly[i][mask] = 1

```

~~~~~^~~~~^

IndexError: list assignment index out of range

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test code crashed before producing data, which means that the pre-registered support conditions could not be unambiguously met. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

## 11. Audit log (LLM calls)

**Total LLM calls:** 10

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 20584        |
| 2  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 18943        |
| 3  | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 10175        |
| 4  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 11015        |
| 5  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 10678        |
| 6  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 14443        |
| 7  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10112        |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 16021        |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 22941        |
| 10 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 8938         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 143850 ms total latency. Provider mix: {'ollama\_remote': 10}

(full prompt+response transcripts available in *research/audit/7b09e71afd40.jsonl*)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/7b09e71afd40.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** *research/pvsnp\_sandbox/test\_\*.py* (per-cycle)

**Replay tarball:** *research/replay/7b09e71afd40.tar.gz* (if generated)